

An Efficient Peer-to-Peer File Sharing Exploiting Hierarchy and Asymmetry

Gisik Kwon

Kyung D. Ryu

*Department of Computer Science and Engineering
Arizona State University, Tempe, AZ 85287
{gkwon, kdryu}@asu.edu*

Abstract

Many Peer-to-Peer (P2P) file sharing systems have been proposed to take advantage of high scalability and abundant resources at end-user machines. Previous approaches adopted either simple flooding or routing with complex structures, such as Distributed Hashing Tables (DHT). However, these approaches did not consider the heterogeneous nature of the machines and the hierarchy of networks on the Internet. This paper presents Peer-to-peer Asymmetric file Sharing System (PASS), a novel approach to P2P file sharing, which accounts for the different capabilities and network locations of the participating machines. Our system selects only a portion of high-capacity machines (supernodes) for routing support, and organizes the network by using location information. We show that our key-coverage based directory replication improves the file search performance to a small constant number of routing hops, regardless of the network size.

1. Introduction

In large-scale computer networks, like the Internet, users have been exchanging information by uploading and downloading files. This service has been provided by the traditional client-server computing model, in which all files are placed and searched from a single or a group of dedicated central servers. However, the servers, which are the central points of the system, do not scale well against the rapid growth of users and their files. This client-server model also fails to take advantage of the resources of end-user computers, although personal machines become more powerful, and the communication network becomes faster and more reliable.

P2P computing model has emerged as an alternative to the traditional model. P2P systems can harness various resources of peer machines, such as unused CPU cycles, memories, and storages from end-user machines. In addition, P2P file sharing systems are now one of the most popular Internet applications and have become a major source of the Internet traffic. Among widely used

applications of such systems are instant messaging, audio-video conferencing, and idle cycle harvesting (SETI@home[4]).

The file sharing system differs from the distributed file systems, such as NFS[12] and AFS[13]. We target a system in which people can publish and share their files with other users, including musics, photos, and regular text files. Distributed file systems are rather intended for transparent file access from remote machines. Such systems focus on maintaining a certain level of consistency among replicated and shared files[17]. File sharing systems provides only the location information of the requested files and a new copy is returned to the requester. Thus, the updates need not be synchronized or propagated.

Recently, a number of P2P file sharing systems, such as Napster[1], Gnutella[2], and Freenet[3], have been introduced. However, due to their simple search algorithms, they do not guarantee the successful search of an existing file. In contrast, the other set of P2P file sharing system substrates, such as CAN[8], Chord[7], Pastry[5] and Tapestry[6] ensures the discovery of an existing file. Nevertheless, the search time increases in proportion to the number of nodes in the system, and they do not provide a mechanism to exploit locality among peer nodes.

P2P file sharing systems need to address the following issues. First, the system should provide an efficient and effective search mechanism. It should return the location of a requested file with minimal communication and computation, while the discovery of a file should be guaranteed no matter where the file exists. Second, the locality-aware searching is needed to improve the response time and minimize the usage of network bandwidth. Third, the system should support self-adaptation to rapid network growth, in cases that the volume of the participating non-dedicated machines grows dramatically.

To address the above issues, we propose a novel Peer-to-peer Asymmetric file Sharing System (PASS). We named our system PASS because it accounts for the asymmetry of resource capacities in various machines

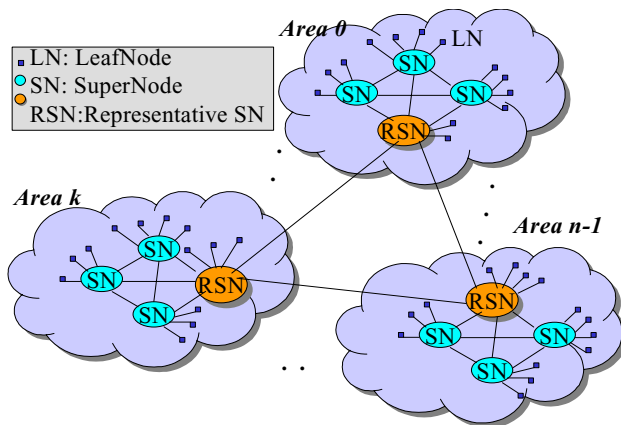


Figure 1: PASS architecture

participating in a peer network. PASS selectively appoints routing-supporting nodes, called supernodes, among only the machines which have a higher capacity than others; whereas previous systems require all the nodes to participate in routing. In this system, a peer network is hierarchically organized with two different types of nodes: supernodes and leafnodes, and with multiple areas. To provide a fast and reliable search, we replicate the file location information (to be called “directory” from now on) over three nodes.

This paper describes the architecture and various mechanisms in PASS. Then, we demonstrate that the system can efficiently search a file with the constant number of routing hops (at most four in the current implementation), which is independent of the number of the participating nodes. PASS also takes full advantage of locality and provides adaptive self-reconfiguration to scale well to the rapid network growth.

This paper is organized as follows: Section 2 provides the overall architecture of PASS, which focuses on the management of areas and supernodes. Section 3 presents the mechanisms for the distribution and replication of file directories. Section 4 explains the dynamic reconfiguration in PASS. Section 5 validates this system by comparing it to another existing P2P file sharing system, and describes the preliminary evaluation results using simulations. Section 6 reviews and discusses the related work. Finally, section 7 concludes with the summary and future work.

2. PASS architecture

PASS divides a peer network into multiple areas. A node is assigned an area using a geographical division, such as a ZIP code, or an administrative network domain. This area division is intended to exploit the geographical or domain-wise locality of the results. All files within the same area are searched first without any interactions with external nodes. This method can reduce the network latency in a file search.

Figure 1 illustrates the hierarchical architecture of PASS, where the asymmetry of the nodes’ capabilities is considered. A previous study demonstrated that a P2P system could be further improved by using a well-designed organization with hierarchy [14]. In our system, hierarchy is used to distribute and replicate directory information of files.

The nodes in PASS, depending on their roles, are divided into two types: leafnodes and supernodes. A leafnode is an ordinary node that can join and leave freely at any time. It can also search, publish, and delete files. A supernode is a selected node that apart from performing normal operations, it participates in locating a specific file. By selectively using only supernodes for distributed directory lookup, we can narrow down the scope of the search space of the file directory. Search requests are forwarded to or exchanged only between supernodes. When a supernode is overloaded with too many leafnode connections or file search requests, it can clone another supernode by converting a relevant leafnode under its management. This processing is also performed when a supernode leaves the system.

A supernode maintains three main tables: a pre_lookup table, a lookup table, and a routing table. The pre_lookup table contains a bitmap, in which each bit indicates whether or not the corresponding file directory is locally available. The lookup table stores the actual file directories. This table contains the condensed file keys, MD5[19] file identifiers, and destination node addresses. The routing table contains the address of the next supernode to which the search will be forwarded.

The special supernode that represents its area is called the Representative SuperNode(RSN). This node is responsible for redirecting search and directory propagation requests from outside to the appropriate supernode in its area. RSNs maintain the location of each other. The consistency of the RSN directory is maintained by using an epidemic protocol. Changes of RSNs can cause expensive directory propagations. To minimize this overhead, we select the most stable node for RSN.

We need to keep supernodes and RSNs highly available since their roles are critical in the system. Therefore, each supernode appoints a backup node among the leafnodes it manages. In cases that the required supernode is not available, the corresponding backup node takes over the role.

3. File directory distribution and replication

In this section, we first discuss the file key management scheme in PASS, then, we describe how the file directory is distributed and replicated. Finally, we discuss our file publishing and searching algorithms.

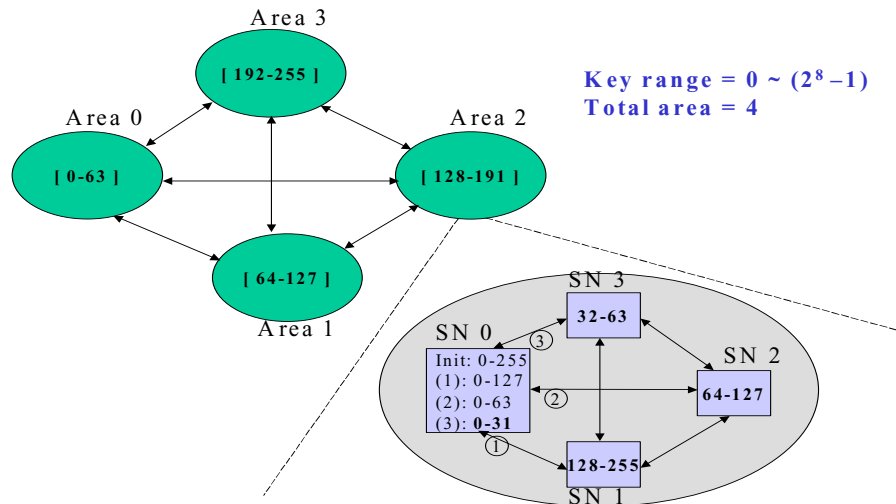


Figure 2: Area Key Coverage(AKC) and Supernode Key Coverage(SKC)

3.1 File key management

In PASS, a file identifier derives from a file name. Then, a file key is condensed from this identifier. A file identifier is used to confirm if the discovered file is exactly the same as the requested one. A file key is used to determine which supernode stores the file directory information. An MD5 message digest is used as a file identifier and it is condensed to a file key. The condensed key is devised to accommodate the pre_lookup table and routing table in a reasonable size. This key is obtained by extracting the predefined number of digits from the file identifier. For example, the file name “let it be” will produce the MD5 file identifier, “78c3465b652724aa-c949527376017cb”. The file key is constructed by using a part of this file identifier. The current implementation adopts the first five digits, 78c34. This condensed key size is selected to allocate only one million entries of memory to the hash-based tables.

The property of balanced distribution in the MD5 message digest is preserved in this condensed key. However, the condensed key can produce more collisions, which can be resolved by the original MD5 file identifier. Our system cannot uniquely identify a file since many files can be published with the same name. In this case, all the files with the same identifier will be returned by PASS and identifying the file is left to users. How this condensed key is used in our system will be explained in Section 3.2 along with the publish and search procedures.

In PASS, a file location record is replicated at three different supernodes for an efficient search. The detailed usage of this directory replication will be explained in Section 3.2. The first copy is placed at the supernode to which the file publisher belongs. For its two replicas, the file key determines their locations. The supernode which

stores the inside-area replica is determined by Supernode Key Coverage (SKC), and the supernode for outside-area replica is determined by Area Key Coverage (AKC). In our system, we assume that the number of areas is set statically at the system initiation. Thus, the AKC for each area is allocated by evenly dividing the key space. Figure 2 demonstrates an example in which the key space is [0, 255]. Since the network has only four areas; each area covers 64 key values. For examples, Area 0 covers from 0 to 63 and Area 1 covers from 64 to 127.

In contrast, SKC is dynamically set as the number of supernodes in an area changes over time. In Figure 2, initially, only one supernode, SN 0 exists in Area 2 that covers the whole key space, from 0 to 255. Whenever a supernode splits into two nodes, its SKC also splits. When SN 0 splits by using another node (SN 1), the key coverage is divided into (0 ~ 127) and (128 ~ 255). Two more subsequent splits of SN 0 reduce the coverage of SN 0 to (0 ~ 31) and create SN 2 and SN 3 with the key coverages (64 ~ 127) and (32 ~ 63) respectively. As shown in this example, SKC may not be evenly divided over all the supernodes. However, this still balances the load when the key values in the original coverage become more popular.

3.2 Directory distribution and file search

In this section, we describe the mechanisms of distributing file directory and locating a file. Both mechanisms take a constant number of steps and use two key coverages, previously defined as AKC and SKC.

A node starts the file publishing procedure by sending a message to the supernode to which it is directly connected. The message contains the following directory record: <file name, file key, file identifier, node IP address>. A user provides only the file name, while the

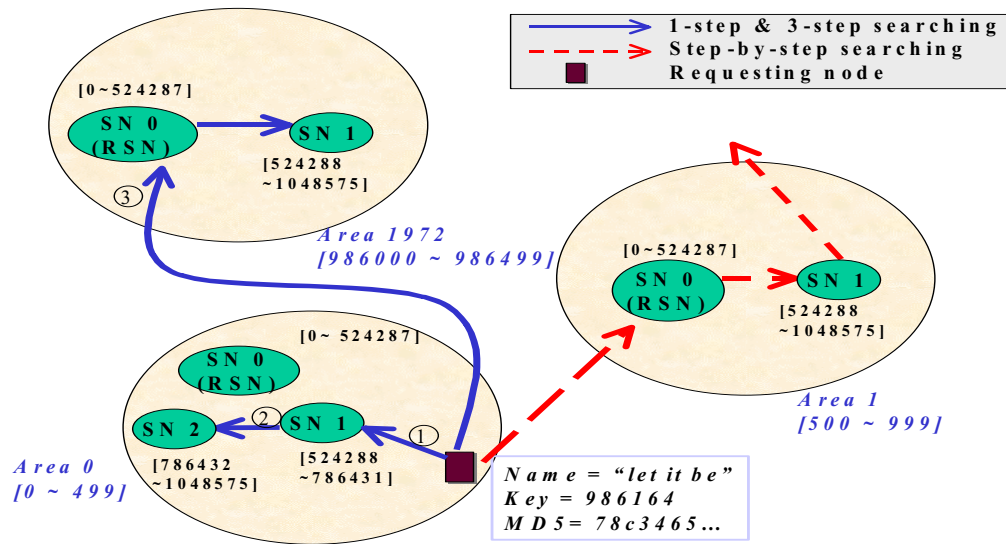


Figure 3: File distribution, replication, and search in PASS

other three values are generated by the system. This directory record is stored in three supernodes. Meanwhile, the file itself remains at the publishing node. Figure 3 demonstrates this process through an example, in which the file key space is 2^{20} (about one million), and the total number of areas is 2097, where each area has 2 or 3 supernodes.

A leafnode in Area 0 publishes a file with the name “let it be.” It first sends the publish request with the directory record to its supernode, SN 1. Then, SN 1 stores the directory record and finds another supernode in the same area, whose SKC covers the file key value. In this example, SN 2 is selected, because its SKC range (786432~1048575) includes the file key 986164. In our implementation, the address of SN 2 can be obtained by reading at most two entries in a routing table. SN2 stores the forwarded directory record in its lookup table.

This directory record is replicated once again by using AKC. The publishing node first computes the area number, whose AKC covers the key value, and then sends the same publish message to the RSN of that corresponding area. The RSN (SN 0 in Area 1972) finds the supernode, whose SKC covers the file key (SN 1), and forwards the request. Finally, SN 1 in Area 1972 stores the directory record of the publish file.

As demonstrated in this example, one file directory is distributed and replicated over three supernodes: 1) the supernode to which it is directly connected; 2) the local supernode, whose SKC covers the file key; and 3) the remote supernode, whose AKC covers the key.

There are three methods to search a file in PASS: the 3-step, 1-step, and step-by-step methods. In the 3-step search method, which is default, a node sends a file search request directly to the connected supernode (the 1st step). The message consists of three fields: <file name, file key,

file identifier>. The supernode first checks its pre_lookup table to quickly determine whether or not the directory of the requested file is available. If the bit is set, the table also includes the index to the lookup table. If the file with the matching digest exists, the supernode returns the destination node address to the requesting node. If there is no such file in the lookup table, or if the bit in the pre_lookup table is unset; the request is transferred to the local supernode with the corresponding SKC (the 2nd step). This local SKC supernode uses its table to search for the file directory. If this search fails, which indicates that the file directory does not exist in that area; the requesting node sends the message to the remote area, whose AKC covers the requested key (the 3rd step). This method fully exploits locality by first searching within its area.

With the 1-step searching method, a node can directly send a request to the remote area. When the file is not popular or it is not expected to be in the local area, this method can save the unnecessary local searching time.

Both methods are subject to failure in case that some areas have no supernodes at the initial stage of the system. In other words, when the remote area, which stores file directory records, based on its AKC, has no supernode, the previous two methods may not find the file; even though the file might exist in another area.

The step-by-step method is used as a backup. To support this method, every RSN needs to maintain the neighbor area information. After obtaining the address of the neighbor RSN, the requesting node sends the search message, along with TTL(Time To Live), to the RSN of its neighbor area. The RSN redirects the message to the appropriate supernode in the area. If the file directory is not found, TTL is decreased by 1, and the request is forwarded to the next area. This procedure is repeated,

until either the file is found or TTL reaches zero. To guarantee the search, we can simply set the TTL to the total number of areas.

4. Self-Organization and dynamic-adaptation

This section introduces a scheme that enables PASS to self-organize when nodes join and leave the system. It also supports dynamic reconfiguration to scale against the growth of the network. It is achieved by dynamically converting leafnodes to supernodes and making them share the load. A new supernode can be created in the following three cases: 1) when the first node joins in an area; 2) when a supernode leaves the system; and 3) when a supernode is overloaded.

When a new node joins PASS; it contacts any existing node to get the RSN of the area. A new node sends a join message to the RSN. A supernode is assigned to the new node in round robin fashion. Finally, the supernode accepts the new node as a leafnode. If there is no RSN in the area, a new supernode needs to be created. In such a case, the joining node becomes the first supernode and the RSN of the area. Its SKC is initialized to cover the whole key space.

In the joining process, there is another case that requires the creation of a supernode. When a supernode manages too many leafnodes, it splits into two by appointing a new supernode. Then, a half SKC range and leafnodes of the current supernode are transferred to the new one.

When a supernode leaves PASS, it needs to create a substitute. The supernode sends a supernode creation request message to the backup node. The message carries all the tables. After the substitute supernode is constructed, the leaving supernode sends the supernode update message to all the supernodes in the area. Then, the supernodes synchronize the supernode address in their routing table. Finally, the leaving supernode requests all its leafnodes to update the supernode address.

A similar procedure takes place when an RSN leaves PASS. In this case, the RSN creates the substitute by converting its backup node. The new RSN sends the RSN update message to all supernodes in the area and RSNs in other related areas.

The last case for creating a new supernode occurs when an existing supernode encounters the system overload. There are various reasons for causing the overload including file directory overflows and excessive incoming search requests. In such cases, a new supernode relieves this load by undertaking half the SKC range and leafnodes from the overloaded supernode. While a new supernode is being created, the existing supernode processes all requests. This is important since it can provide continuous operation during configuration changes.

5. Validation & simulations

In this section, we first analyze the resource requirement of our PASS and compare it with that of a popular Distributed Hashing Table (DHT) based system. We estimate how much memory and computation overhead is required by PASS. Second, using simulations, we investigate the supernode creation rate for different network usage limits. Third, we compare the search performance as a peer network grows over time. To our knowledge, full-scale trace data from existing P2P systems is not yet available. Thus, we conduct simulations with various sizes of network.

5.1 Validation

We compare resource requirements between our PASS and the Chord system, a DHT-based file sharing system based on consistent hashing [18]. The Chord guarantees the discovery of a requested file in $\log_2 N$ routing hops and requires $\log_2 N$ routing table size, where N is the total number of nodes. Most of the other DHT-based systems have a similar architecture. We consider a fairly large network consisting of one million nodes. In PASS, the network is comprised of 100 areas; each area contains 100 supernodes; and each supernode owns 100 leafnodes. We assume that the nodes are evenly distributed in Chord.

First, we want to see how much memory is required for each system. The exact amount can be different depending on the implementation of data structures. Thus, we take the number of entries in routing tables as the measure of memory overhead. For one million nodes, Chord requires 20 entries for each routing table. In contrast, the routing table size in PASS is not affected by the network size, but is rather determined by the condensed key size. As described in Section 3.1, the current condensed key size of 5 hexadecimal digits yields one million entries in a routing table.

In PASS, only supernodes, which are merely 1% of all nodes in this case, maintain routing tables; whereas Chord needs routing tables at all the nodes. We believe that allocating one million entries (conservatively, 20 Mbytes) from selected resource-superior machines is quite acceptable. Today, a decent personal computer contains between 256 Megabytes and 1 Gigabytes of memory. The PASS routing table requires less than 10% of the total memory of such machines.

The second analysis is concerned with the computational overhead of file searching. Total computations for a search can be computed using the number of nodes to be visited and the routing table lookup time at each node. Both Chord and PASS take a constant time for a routing table lookup, since each system uses hash-based tables. Thus, we estimate the maximum number of routing nodes to visit for a search. In Chord, this number can vary for different system sizes. In cases of 1, 10, and 100-million nodes, Chord needs 20, 24, and

Event	Leafnode join	File search	File publish	File delete
λ^*	1.0	1.0	0.01	0.001

* Average occurrence rate (unit time : hour)

Table 1: Event occurrence rates in Poisson distribution

27 routing hops respectively according to its $\log_2 N$ search performance. However, PASS always finishes a search with at most 4 hops as described in Section 3.2. In the one million nodes case, PASS will reduce the lookup overhead to 20% of that of Chord.

In summary, PASS requires more but an acceptable size of memory to store routing tables. However, PASS imposes much less computation overhead thanks to the reduced node visits for a search.

5.2 Simulations

First, we simulate a PASS network to estimate how much network bandwidth is required for a supernode as more nodes join the system. We also present how many and frequently supernodes are created, when the threshold of network bandwidth for a supernode is set. This metric is important because supernode creation is a quite expensive operation in PASS.

The event occurrence rates follow the Poisson distribution. The average occurrence rate for each event is presented in Table 1. The estimated bandwidth and computation overhead for each operation are summarized in Table 2. The internal resource limits, such as the size of the file lookup table and the number of leafnodes for a supernode, are not bounded.

In Figure 4, the curve demonstrates the bandwidth increase as more nodes join over time. It shows that the bandwidth requirement increases linearly, and reaches 4.7 Mbytes/sec at the point of 10,000 hours. We also investigate how frequently supernodes are created to relieve the overload. Figure 5 illustrates two curves that represent the bandwidth limits of 0.5 Mbytes/sec and 1.5 Mbytes/sec, respectively. We can observe that the system in this simulation setup creates a supernode every 600 hours for the first case and every 2000 hours for the second. This simulation shows that PASS does not exceed the negotiated network bandwidth limit by creating more supernodes adaptively.

We conducted another simulation to see how a file search performs in PASS. In this simulation, the 3-step searching method is used first. When this method fails, the step-by-step searching method is used. This simulation contains 100 areas, and uses the same event rates as in Table 1.

Figure 6 presents the search performance for PASS in the number of network hops. The curve of $\log_2 N$ represents the approximate performance of Chord. As illustrated in Figure 6, the curve for PASS shows three phases: Cold phase, Warm phase, and Hot phase. In the

Node	Event	Bandwidth*	Computation**
RSN in the same area	Join	1[4]+1[28]	1[1]
	Search	1[4]+1[24]	1[1]
	publish	1[4]+1[24]	1[1]
SN directly connected	Join	1[32]+1[4]	1[1]
	Search	1[68]+1[1]	1[1]
	publish	2[97]+2[97]	3[4]
SN covering the SKC in the same area	Join	-	-
	Search	1[68]+1[1]	1[1]
	publish	2[97]+2[97]	2[2]
RSN covering the AKC	Join	-	-
	Search	1[68]+1[96]	1[2]
	publish	1[96]+1[96]	1[2]
SN covering the SKC in the remote area	Join	-	-
	Search	1[96]+1[28]	3[21]
	publish	1[96]+1[96]	2[2]

*Bandwidth = # incoming_message[size] + # outgoing_message[size] (size : byte)

**Computation = # accessed table[# lookup]

Table 2: The costs of operations in PASS

Cold phase, a search requires much higher network hops (up to 90). This is due to the empty areas, in which no supernode has yet been created, and their AKCs cannot be handled. In this case, the 3-step method often fails and the expensive step-by-step mechanism is used. However, we can easily assume that each area will have at least one node at system initiation. This assumption will remove the costly Cold phase.

Later, the system enters the second phase, the Warm phase. Since all the areas now operate with one or more supernodes in most of the time, the 3-step mechanism finds the requested file within 4 hops (3.8 on the average). As more nodes join and more files are published, the system gets saturated and enters the Hot phase. In this phase, a large fraction of file searches are answered locally inside the area and the average routing hops drops even lower (1.8). In conclusion, once PASS reaches the warm phase, it shows the good search performance with the constant number of routing hops.

6. Related work

In peer-to-peer file sharing systems, a straightforward way to achieve a good search performance is to put all the directory files into a central server. Napster[1] follows this approach. A client can simply visit this central directory server and find the location of a desired file. This system can be called a P2P system, in the sense that, files are stored at client machines, not at a dedicated server. However, this system has some serious drawbacks: limited scalability and a single point of failure[15]. PASS addresses these problems by distributing and replicating the directory over multiple areas and multiple directory supporting nodes (supernodes).

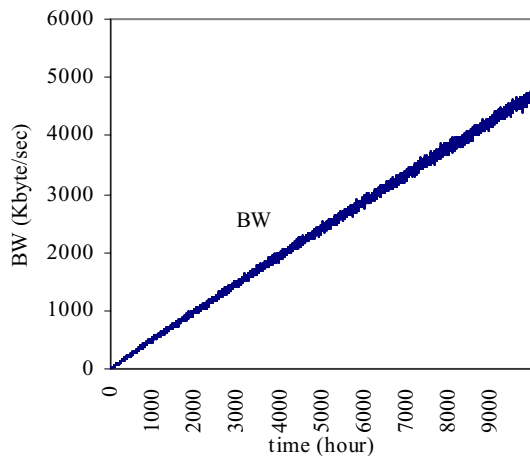


Figure 4: The bandwidth usage

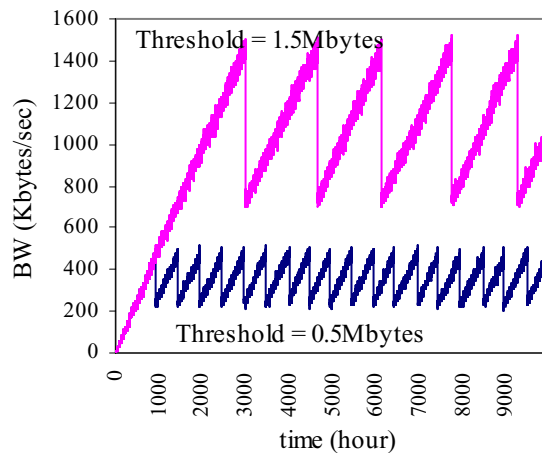


Figure 5: The supernode creation rate

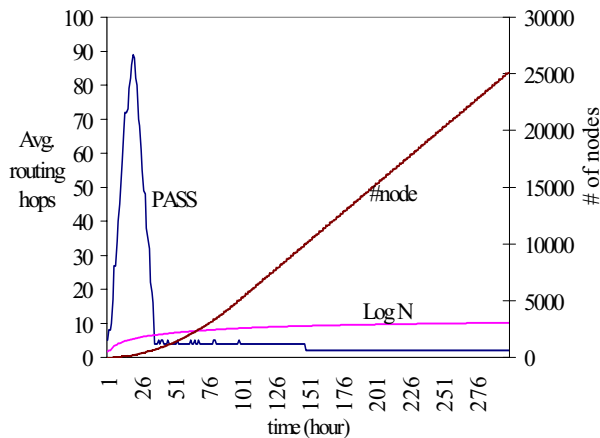


Figure 6: The search performance

With respect to distributing a global directory, PASS and DNS[11] adopt similar mechanisms. However, in DNS, name servers cannot be dynamically and adaptively created, while supernodes in PASS can be.

Currently, several pure P2P file sharing systems are in use. Such systems include Gnutella[2] and Freenet[3]. These systems provide a large-scale distributed file sharing service. Gnutella searches a file by using the broadcast-based protocol that restricts the scalability and imposes a high network bandwidth overhead. Freenet forwards and backtracks the search request, based on the routing table. Nevertheless, these systems have a major drawback: they cannot guarantee that a file will be found if it exists somewhere in the system[15].

To overcome this problem, new P2P routing and searching mechanisms have been introduced. Systems, such as Pastry[5], Tapestry[6], Chord[7], and CAN[8] provide guaranteed location of an existing file. The Plaxton algorithm[9] and the landmark hierarchy[10] are

the basis of the search schemes in all these systems. Both Pastry and Tapestry show the same search performance of $O(\log_2^b N)$, in which b is a configuration parameter. However, they use different approaches to routing. Chord routes messages, based on the numerical difference between the current node ID and the destination ID. Chord provides the $O(\log_2 N)$ search performance. CAN routes messages in a d -dimensional space, where each node maintains a routing table with $O(d)$ entries and any node can be reached in $O(d \cdot N^{1/d})$ routing hops. In contrast, PASS uses only selected nodes for routing (supernodes).

Recently, Yang et al. have stressed the use of the super-peers to cope with the heterogeneity of the capabilities across peers and have empirically discovered a set of designing rules for the super-peer network[16].

7. Conclusion and future work

This paper presented the design and preliminary evaluation of PASS, a novel peer-to-peer distributed file sharing system. Our system provides a self-organized, adaptive, and scalable distributed file searching and routing. PASS can publish and search a file with the constant routing hops (at most four in the current implementation). PASS divides the network into autonomous areas so that the file directory can be distributed and replicated over the areas. The routing information is also replicated for a fast local area search. Our preliminary evaluation with simulations demonstrates that PASS can provide the efficient search performance after a system warming period.

PASS has been implemented in Java with SDK1.4.0. It uses RMI and the asynchronous message passing technology.

We are currently improving PASS to provide fast failure detection and recovery mechanisms for supernodes. We are investigating to find the proper size for the

condensed key. Efficient disk swapping is necessary in case of large table management. Currently, PASS can search only the files, which contain the exactly matching file key. We are investigating on extending PASS to provide the keyword-based searching. Finally, we plan to study the security implications of PASS.

References

- [1] Napster, <http://www.napster.com/>, 2000.
- [2] Gnutella, "The Gnutella Protocol Specification", <http://dss.clip2.com/GnutellaProtocol04.pdf>, 2000.
- [3] I. Clarke, O. Sandberg, B. Wiley, and T. Hong, "Freenet: A Distributed Anonymous File Storage and Retrieval System", Workshop on Design Issues in Anonymity and Unobservability, 2000.
- [4] SETI@home, <http://setiathome.ssl.berkeley.edu/>, 1996.
- [5] Rowstron and P. Druschel, "Pastry: Scalable, Distributed File Location and Routing for Large-Scale Peer-to-Peer Systems", IFIP/ACM International Conference on Distributed Systems Platforms (Middleware), Heidelberg, Germany, November 2001, pp. 329-350.
- [6] B.Y. Zhao, J.D. Kubiatowicz, and A.D. Joseph, "Tapestry: An Infrastructure for Fault-tolerant Wide-Area Location and Routing", U.C.Berkeley Technical Report UCB//CSD-01-1141, April 2000.
- [7] I. Stoica, R. Morris, D. Karger, M.F. Kaashoek, and H. Balakrishnan, "Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications", ACM SIGCOMM 2001, San Diego, CA, August 2001, pp. 149-160.
- [8] S. Ratnasamy, P. Francis, M. Handley, R. Karp, and S. Shenker, "A Scalable Content-Addressable Network", Proc. ACM SIGCOMM 2001, August 2001.
- [9] C.G. Plaxton, R. Rajaraman, and A.W. Richa, "Accessing Nearby Copies of Replicated Files in a Distributed Environment," Theory of Computing Systems, 1999.
- [10] P.F. Tsuchiya, "The Landmark Hierarchy: A New Hierarchy for Routing in Very Large Networks", SIGCOMM'88, Stanford, CA, 1988.
- [11] P. Mockapetris, "Domain Names – Implementation and Specification," RFC-1035, Network Working Group, November 1987.
- [12] R. Sandberg, et al, "Design and Implementation of The Sun Network Filesystem", Proceedings of the USENIX 1985 Summer Conference, June 1985, pp. 119-130.
- [13] M. Satyanarayanan, et al, "The ITC Distributed File System: Principles and Design", Proceedings of the 10th Symposium on Operating Systems Principles, December 1985, pp. 35-50.
- [14] B.Y. Zhao, Y. Duan, L. Huang, A. Joseph, and J. Kubiatowicz, "Brocade: Landmark Routing on Overlay Networks", Proceedings of the First International Workshop on Peer-to-Peer Systems (IPTPS 2002), March 2002.
- [15] Y. Beverly, G.M. Hector, "Comparing Hybrid Peer-to-Peer Systems", Technical report, Stanford University Stanford, 2000.
- [16] Y. Beverly, G.M. Hector, "Designing a Super-Peer Network", Technical report, Stanford University Stanford, 2002.
- [17] T.W. Page, R.G. Guy, J. S. Heidemann, D.H. Ratner, P.L. Reiher, A. Goel, G.H. Kuenning, and G. Popek. "Perspectives on Optimistically Replicated Peer-to-Peer Filing", Software Practice and Experience, December 1997.
- [18] D. Karger, "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web", ACM Symposium on Theory of Computing, 1997.
- [19] M.J.B. Robshaw, "MD2, MD4, MD5, SHA and Other Hash Functions", Technical Report TR-101, version 4.0, RSA Laboratories, 1995.